



St. MARTIN'S ENGINEERING COLLEGE

(Autonomous Institution - UGC, Govt. of India)

| ESTD : 2002 | NAAC (A+) & NBA Accredited | Affiliated to JNTUH
| Approved by AICTE | NIRF & ARIIA Ranked | A Non Minority Institution
Dhulapally, Near Kompally, Secunderabad - 500 100. Telangana.



ASSIGNMENT ANSWER BOOKLET

Hall Ticket No. : 24K81A05D3

Name of the Student: A. Deekshitha

Course : B-Tech

Branch : CSE

Year : I/II/III/IV

Sem : I/II

Assignment : I/II

Subject Name : Design and Analysis of Algorithms (DAA)

Last Date of Submission:

Actual Date of Submission :

MARKS AWARDED

Q.No.	a	b	Total
1			
2			
3			
4			
5			
Total			

A. Deekshitha

Signature of the Student

Signature of the Evaluator

Start Writing from Here

1. a) Explain the process of designing an algorithm. Give characteristics of an algorithm.

ALGORITHM :

- An Algorithm is a set of steps of operations to solve a problem performing calculations, data processing and automated reasoning tasks.
- An algorithm is an efficient method that can be expressed within finite amount of Time and space.
- The important aspect of algorithm design include creating an efficient algorithm to solve a problem in an efficient way using minimum time and space.
- To solve a problem, different approaches can be followed.

Some of them can be efficient with respect to time consumption, whereas other approaches may be memory efficient.



Process of designing an algorithm:

Designing an algorithm is a structured process of defining a solution to a problem before implementing it in code.

The steps involved in this process are:

- 1) Understand the problem
→ Clearly define the problem and identify inputs, outputs and constraints.
- 2) Analyze the requirement.
→ Determining what the algorithm needs to accomplish; special cases, edge cases and identifying performance requirements.
- 3) Develop a solution strategy.
→ Breaking the problem into smaller subprograms and choosing an appropriate approach, such as: Brute force, Greedy method.
- 4) Design the algorithm.
→ Writing the steps in plain language, pseudocode or a flow chart to ensure the logic is clear and complete.
- 5) Verify the algorithm.
→ Tracing the algorithm using sample inputs and checking that it produces the correct outputs. Test cases, boundary cases & invalid input.
- 6) Analyze efficiency.
→ Evaluate the time complexity and space complexity and optimize if necessary.
- 7) Implement the algorithm
→ Translate the algorithm into a programming language and follow good coding practices for readability and maintainability.
- 8) Test and refine.
→ Running the program with test cases and fixing bugs and improve performance if needed and updating the algorithm if requirement changes.



Characteristics of an algorithm

- 1) INPUT: The algorithm should be given zero or more input.
- 2) OUTPUT: At least one quantity is produced. For each input the algorithm produced value from specific task.
- 3) DEFINITENESS: Each instruction is clear and unambiguous.
- 4) FINITENESS: If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after a finite number of steps.
- 5) EFFECTIVENESS: Every instruction must be very basic so that it can be carried out, in principle, by a person using only pencil and paper.

(b) Explain asymptotic notations?

ASYMPTOTIC ANALYSIS -

- Expressing the complexity in terms of its relationship to know function. This type analysis is called asymptotic analysis.
- The main idea of Asymptotic analysis is to have a measure of efficiency of an algorithm, that doesn't depends of.

1. Machine constants
2. Doesn't require algorithm to be implemented.
3. Time taken by program to be prepared.

ASYMPTOTIC NOTATION -

- The mathematical way of representing the Time complexity.
- The notation we use to describe the asymptotic running time of an algorithm are defined in terms of functions whose domains are the set of natural numbers.

Defination: It is the way to describe the behaviour of functions in the limit or without bound.

Asymptotic growth:

The rate at which the function grows "growth rate" is the complexity of the function or the amount of resource it takes up to compute.



Growth rate $\longrightarrow$ Time + memory.

Q

1) Illustrate the general method of divide and conquer technique.

Divide and conquer technique:

It is an algorithm design technique in which a large problem is divided into smaller subproblems, each subproblem is solved separately, and then their solutions are combined to get the final solution.

General method of divide and conquer technique:

Algorithm D And C(P)

{
if small(P) then return S(P);

else

{

divide P into smaller instances

$P_1, P_2, \dots, P_k, k \geq 1;$

Apply D and C to each of these sub problems;

return combine (D And C(P_1), D And C(P_2), ..., D And C(P_k));

}

}

Example:

Merge sort

We have [8, 3, 5, 2, 7, 1]

Divide:- [8, 3, 5] [2, 7, 1]

divide again: [8] [3, 5] [2] [7, 1]



continue until individual element is obtained.

[8] [3] [5] [2] [7] [1]

conquer:- Sort the small parts:

[3, 5] [1, 7]

combine: Merge the sorted parts

[3, 5, 8] + [1, 2, 7]

↓

[1, 2, 3, 5, 7, 8]

b) write an algorithm for Binary search and discuss its complexity.

Algorithm:

Algorithm BinSearch (a, n, x)

// Given an array $a[1:n]$ of elements in non-decreasing order, $n \geq 0$, determine whether x is present and if so, return.

// j such that $x = a[j]$; else return 0.

```
{
  low := 1; high := n;
  while (low <= high) do
  {
    mid :=  $\lfloor (low + high) / 2 \rfloor$ ;
    if ( $x < a[mid]$ ) then high = mid - 1;
    else if ( $x > a[mid]$ ) then low = mid + 1;
    else return mid;
  }
```

```
}
return 0;
}
```

Time complexity:

$$T(n) = T\left(\frac{n}{2}\right) + 1 \quad \text{--- (1)}$$



Substitute $n/2$ in eq ①:

$$T\left(\frac{n}{2}\right) = T\left(\frac{n/2}{2}\right) + 1$$

$$T\left(\frac{n}{2}\right) = T\left(\frac{n}{4}\right) + 1 \text{ --- ②}$$

Substitute $T(n/2)$ value in eq ①

$$T(n) = [T\left(\frac{n}{4}\right) + 1] + 1$$

$$T(n) = T\left(\frac{n}{4}\right) + 2 \text{ --- ③}$$

Substitute $n/4$ in eq ①

$$T\left(\frac{n}{4}\right) = T\left(\frac{n/4}{2}\right) + 1$$

$$T\left(\frac{n}{4}\right) = T\left(\frac{n}{8}\right) + 1 \text{ --- ④}$$

Substitute $T(n/4)$ value in eq ③.

$$T(n) = [T\left(\frac{n}{8}\right) + 1] + 2$$

$$T(n) = T\left(\frac{n}{8}\right) + 3 \text{ --- ⑤}$$

Let $k=3, n=2^k$. So $k=\log_2 n$

Substitute k and n in eq ⑤

$$T(n) = T\left(\frac{n}{2^k}\right) + k$$

$$T(n) = T\left(\frac{n}{n}\right) + k$$

$$T(n) = T(1) + \log_2 n \Rightarrow T(n) = O(\log n)$$

[$\because T(1)$ is constant]

- 3 Discuss the single-source shortest paths (i.e. Dijkstra's algorithm) with suitable example and also find the time complexity.

Dijkstra's algorithm: It is a greedy algorithm used to find the shortest path from a single source vertex to all other vertices in a weighted graph with non-negative edge weights.

Algorithm:

Algorithm Shortest Paths ($v, cost, dist, n$)

{



```

for i := 1 to n do
  s[i] := false ; dist[i] := cost[v, i];
}
s[v] := true ; dist[v] := 0
for num := 2 to n-1 do
  {
    s[u] := true;
    for (each w adjacent to u with s[w] = false) do
      if (dist[w] > dist[u] + cost[u, w]) then
        dist[w] := dist[u] + cost[u, w];
      }
    }
  }

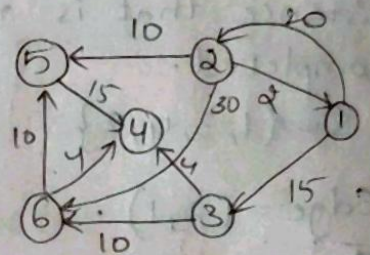
```

Example:

1. Read the edges

- (i) (1, 2) → 20
- (ii) (2, 1) → 2
- (iii) (1, 3) → 15
- (iv) (3, 6) → 10
- (v) (3, 4) → 4

- (vi) (6, 5) → 10
- (vii) (5, 4) → 15
- (viii) (6, 4) → 4
- (ix) (2, 5) → 10
- (x) (2, 6) → 30



2. $S = \{1\}$

dist[1] = 0

Edge = (1, 2) - 20

if (dist[w] > dist[u] + cost[u, w])

then dist[w] = dist[u] + cost[u, w]

if (dist[2] > dist[1] + cost[1, 2])

$\infty > 0 + 20 \rightarrow \text{True}$

dist[2] = 20

Edge = (1, 3) - 15

if (dist[3] > dist[1] + cost[1, 3])

$\infty > 0 + 15 \rightarrow \text{True}$

dist[3] = 15

vertex 3 is selected. As 1 → 3 has the shortest distance.

vertex	dist	path
1	0	-
2	20	-
3	15	-
4	∞	-
5	∞	-
6	∞	-



It is added to 'S'

$$S = \{1, 3\}$$

$$\text{Edge} = (3, 4) - 4$$

if $(\text{dist}[4] > \text{dist}[3] + \text{cost}[3, 4])$

$$\infty > 15 + 4 \rightarrow \text{True}$$

then $\text{dist}[4] = 19$

$$\text{Edge} = (3, 6) - 10$$

if $(\text{dist}[6] > \text{dist}[3] + \text{cost}[3, 6])$

$$\infty > 15 + 10 \rightarrow \text{True}$$

$$\text{dist}[6] = 25$$

vertex 4 is selected & $S = \{1, 3, 4\}$

As 4 does not have any outgoing edges. So, see the next least distance that is not yet completed.

$$S = \{1, 3, 4, 2\}$$

$$\text{Edge} = (2, 1)$$

As 1 already exists in S.

Ignore this edge.

$$\text{Edge} = (2, 5) - 10$$

if $(\text{dist}[5] > \text{dist}[2] + \text{cost}[2, 5])$

$$\infty > 20 + 10 \rightarrow \text{True}$$

then $\text{dist}[5] = 30$

$$\text{Edge} = (2, 6) - 30$$

if $(\text{dist}[6] > \text{dist}[2] + \text{cost}[2, 6])$

$$25 > 20 + 30 \rightarrow \text{False}$$

vertex 6 is selected and

$$S = \{1, 3, 4, 2, 6\}$$

$$\text{Edge} = (6, 4)$$

As 4 already exists in 'S'.

Ignore this edge.

$$\text{Edge} = (6, 5) - 10$$

if $(\text{dist}[5] > \text{dist}[6] + \text{cost}[6, 5])$

$$30 > 25 + 10 \rightarrow \text{False}$$

Since the condition failed. Let's consider next least distance that is not yet completed.

$$S = \{1, 3, 4, 2, 6, 5\}$$

$$\text{Edge} = (5, 4)$$

As 4 already exists in 'S'.

Ignore this edge.

Since, all the edges are visited then we written return 0.

vertex	dist	path
1	0	1
2	20	1 → 2
3	15	1 → 3
4	19	1 → 3 → 4
5	30	1 → 2 → 5
6	25	1 → 3 → 6



Time complexity:

- The first for loop runs n times $\rightarrow O(n)$.
- The outer loop from line 8 runs $n-1$ times $\rightarrow O(n)$
- Inside it finding minimum-distance vertex takes $\rightarrow O(n)$
- Updating distances also takes $\rightarrow O(n)$.
- Therefore $O(n) \times O(n) = O(n^2)$

4 Find an optimal solution to the knapsack instances $n=7$ objects and capacity of knapsack $m=15$. The profits and weights of the objects were

$$(P_1, P_2, P_3, P_4, P_5, P_6, P_7) = (10, 5, 15, 7, 6, 18, 3);$$

$$(w_1, w_2, w_3, w_4, w_5, w_6, w_7) = (2, 3, 5, 7, 1, 4, 1).$$

Given n objects & a knapsack where object i has a weight w_i & the knapsack has a capacity m .

- If a fraction x_i of object i placed into knapsack, a profit $P_i x_i$ is earned.
- The objective is to obtain a filling of knapsack maximizing the total profit.

$$\text{maximize } \sum_{1 \leq i \leq n} P_i x_i$$

$$\text{Subject to } \sum_{1 \leq i \leq n} w_i x_i \leq m$$

$$\& \ 0 \leq x_i \leq 1; \ 1 \leq i \leq n$$

Knapsack (m, n)

$u \rightarrow$ after placing how much is remaining space is available.

$X[i] \rightarrow$ Product placed in bag increment by 1.



$w[i] \rightarrow$ weight of bag.

knapsack instances $n=7$ objects.

Capacity of knapsack $m=15$.

$$(P_1, P_2, P_3, P_4, P_5, P_6, P_7) = (10, 5, 15, 7, 6, 18, 3)$$

$$(w_1, w_2, w_3, w_4, w_5, w_6, w_7) = (2, 3, 5, 7, 1, 4, 1)$$

$$(P_1/w_1, P_2/w_2, P_3/w_3, P_4/w_4, P_5/w_5, P_6/w_6, P_7/w_7)$$

$$= (5, 1.6, 3, 1, 6, 4.5, 3)$$

i	1	2	3	4	5	6	7
P	6	10	18	15	3	5	7
w	1	2	4	5	1	3	7
P/w	6	5	4.5	3	3	1.6	1

i	u	w[i]	x[i]
1	15	1	1
2	14	2	1
3	12	4	1
4	8	5	1
5	3	1	1
6	2	3	0.6

$$\text{Profit} = 6 + 10 + 18 + 15 + 3 + (5 \times 0.6) = 55$$

5. Draw an optimal Binary search tree for $n=4$ identifiers $(a_1, a_2, a_3, a_4) = (\text{do}, \text{if}, \text{read}, \text{while})$.
 $P(1:4) = (3, 3, 1, 1)$ and $Q(0:4) = (2, 3, 1, 1, 1)$.

Given

$n=4$

identifiers:

$$(a_1, a_2, a_3, a_4) = (\text{do}, \text{if}, \text{read}, \text{while})$$



$$P(1:4) = (3, 3, 1, 1)$$

$$Q(0:4) = (2, 3, 1, 1, 1)$$

For an optimal Binary search tree, we calculate the weight, cost and root of each subtree.

The formula for cost is:

$$C(i, j) = \min_{i \leq k \leq j} [C(i, k-1) + C(k+1, j) + w(i, j)]$$

$$\text{where } w(i, j) = w(i, j-1) + P_i + Q_j$$

Step-1: Root table

After calculating the costs, the root values are

	1	2	3	4
1	1	1	1	1
2		2	2	2
3			3	3
4				4

For the complete set 1 to 4:

$$\text{Root}(1, 4) = 2$$

So, $a_2 = \text{if}$ is the root.

Step-2: left subtree

for 1 to 1

$$\text{Root}(1, 1) = 1$$

So, $a_1 = \text{do}$ becomes the left child of it.

Step-3: Right subtree

for 3 to 4

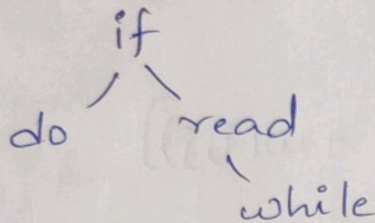
$$\text{Root}[3, 4] = 3$$

So, $a_3 = \text{read}$ becomes the right child of it.



The remaining node a_4 = while becomes the right child of read.

Optimal Binary search tree.



∴ The optimal BST is,

- Root $\rightarrow$ if
- Left child $\rightarrow$ do
- Right child $\rightarrow$ read
- Right child of read $\rightarrow$ while

The minimum cost of the optimal BST is 40

minimum cost = 40